

01 AOU Specification

- [AOU CORE — Specification \(LT\)](#)
 - [1. Purpose](#)
 - [2. Software-visible behaviour](#)
 - [2.1 Register map \(offsets\)](#)
 - [2.2 Activation \(`aou\_init`\)](#)
 - [2.3 AXI bridge](#)
 - [2.4 Interrupt](#)
 - [3. Out of scope \(this model\)](#)

AOU_CORE — Specification (LT)

1. Purpose

`aou_core` is a loosely-timed SystemC/TLM model of the AXI-over-UCle (AOU) bridge. It is for firmware bring-up and VP integration — not cycle-accurate FDI/flit simulation.

Ground truth for software-visible CSRs: [aou-rtl/csr/aou-core.rdl](#).

SMC VP placement: base `0xC000_4000`, window `0x80`.

2. Software-visible behaviour

2.1 Register map (offsets)

Offset	Name	Access	Reset	LT role
0x00	<code>ip_version</code>	R	<code>0x00010000</code>	Major=1, minor=0
0x04	<code>aou_con0</code>	RW (+ WO bit0)	<code>0x00004008</code>	Soft reset (bit4); <code>deactivate_force</code> (bit0, WO)
0x08	<code>aou_init</code>	RW / R / W1C	see §2.2	Activate / deactivate / status / ints
0x0C	<code>aou_interrupt_mask</code>	RW	0	Storage only — see §2.4
0x10	<code>lp_linkreset</code>	RW	<code>0x00002400</code>	Storage only
0x14	<code>dest_rp</code>	RW	<code>0x00003210</code>	Per-RP destination for AXI bridge
0x18	<code>prior_rp_axi</code>	RW	<code>0x0A503210</code>	Storage only
0x1C	<code>prior_timer</code>	RW	<code>0x000F000F</code>	Storage only
0x20–0x7C	RP0...RP3 banks	RW	split= <code>0x0F0F</code> , else 0	Storage only (6×4 regs)

2.2 Activation (`aou_init`)

Bit	Field	Behaviour
0	<code>activate_start</code>	Write 1 to request enable; clears when ENABLED
1	<code>deactivate_start</code>	Write 1 to request disable
2	<code>activate_state_enabled</code>	R — link ENABLED
3	<code>activate_state_disabled</code>	R — link DISABLED (reset default)
6:4	<code>deactivate_time_out_value</code>	RW storage
7	<code>int_deactivate_start</code>	Sticky; W1C
8	<code>int_activate_start</code>	Sticky; W1C
10:9	<code>mst/slv_tr_complete</code>	R — always 1 in LT

Enable rules

1. `fdi_active_i` must be true.
2. Writing `activate_start` enables the local die.
3. If a peer is connected via `connect_peer()`, the peer auto-acks (LT: remote die already ready). Firmware programs only the local CSRs.

Disable: `deactivate_start` or `aou_con0.deactivate_force`.

2.3 AXI bridge

When **both** local and peer are ENABLED, a transaction on local `axi_s[rp]` is forwarded to peer `axi_m[dest]`, where `dest = dest_rp[rp]` (2 bits per RP). Otherwise the slave returns `TLM_COMMAND_ERROR_RESPONSE`.

2.4 Interrupt

`irq_o = int_activate_start || int_deactivate_start`. Cleared by W1C on those bits.

In the SMC VP, `irq_o` is aggregated into `interrupt_aggregator (intagg)` alongside every other SMC peripheral and drives **PLIC source ID 27** (`src_in[26]`) — firmware enables/claims it through the PLIC the same way it does for UART/I3C/I2C/telemetry, rather than only being observable by polling `aou_init` directly.

`aou_interrupt_mask` is **storage-only** in this LT model, and that is a faithful match to the real RTL, not a shortcut: per `aou-core.rdl`, the mask's 7 bits gate the linkreset-ACK-timeout, invalid-actmsg, msgcredit-timeout, early-response, and MI0/SI0 ID-mismatch interrupts — none of which this LT model implements (see §3, “out of scope”). `int_activate_start` / `int_deactivate_start` are **not** covered by any mask bit in the real hardware either, so they are unconditional in `AOU_CORE.sv` as well as here. If a future revision adds any of the masked error sources, `update_irq()` must be extended to AND them against `aou_interrupt_mask` before OR-ing into `irq_o` — today there is nothing for the mask to gate.

3. Out of scope (this model)

- FDI flit packing / unpacking, credits, stall, cancel
- Early BRESP, AXI split/aggregator datapaths, QoS arbitration timing
- FDI bringup FSM (`AOU_FDI_BRINGUP_CTRL`)